

SULIT



SECP2523 DATABASE

SESSION 2024/2025 - SEMESTER 1

ALTERNATIVE ASSESSMENT REPORT: PHASE 2

Name	TAN ZHI MING
IC No. / Matric No.	041118011233
Year / Program	2/SECPH
Section	2
Lecturer Name	DR SEAH CHOON SEN
Case Study	Administrative Module
System Name	KADA-eServe
Group Name	TechAway

SECTION C (SQL AND INTERFACE IMPLEMENTATION)

3.1 SQL Statement (DDL & DML)

DDL:

1. Create Transactions Table

```
CREATE TABLE transactions (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  member_id BIGINT UNSIGNED NOT NULL,  
  type ENUM('savings', 'loan') NOT NULL,  
  savings_type VARCHAR(255) NULL,  
  loan_id VARCHAR(50) NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NULL,  
  FOREIGN KEY (member_id) REFERENCES member_register(id),  
  FOREIGN KEY (loan_id) REFERENCES loans(loan_id)  
);
```

SELECT * FROM `transactions`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

id	member_id	type	savings_type	loan_id	amount	created_at	updated_at
----	-----------	------	--------------	---------	--------	------------	------------

↑ ☐ Check all With selected: Edit Copy Delete Export

2. Create Loans Table

```
CREATE TABLE loans (  
  loan_id VARCHAR(50) PRIMARY KEY,  
  member_id BIGINT NOT NULL,  
  loan_type_id BIGINT NOT NULL,  
  bank_id BIGINT NOT NULL,  
  date_apply DATE,  
  loan_amount DECIMAL(10,2),  
  interest_rate DECIMAL(5,2),  
  monthly_repayment DECIMAL(10,2),  
  monthly_gross_salary DECIMAL(10,2),  
  monthly_net_salary DECIMAL(10,2),  
  loan_period INTEGER,  
  status VARCHAR(20) DEFAULT 'pending',
```

```

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP,
FOREIGN KEY (member_id) REFERENCES member_register(id),
FOREIGN KEY (loan_type_id) REFERENCES loan_types(loan_type_id),
FOREIGN KEY (bank_id) REFERENCES banks(bank_id)
);

```

```
SELECT * FROM `loans`
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

```
loan_id member_id loan_type_id bank_id date_apply loan_amount interest_rate monthly_repayment monthly_gross_salary monthly_net_salary loan_period status created_at updated_at
```

DML:

1. SELECT: Get pending loan applications with member details for approval

```

SELECT l.loan_id,
       m.name AS member_name,
       m.ic AS member_ic,
       lt.loan_type,
       l.loan_amount,
       l.monthly_gross_salary,
       l.monthly_net_salary,
       l.monthly_repayment,
       l.date_apply,
       b.bank_name,
       b.bank_account
FROM loans l
JOIN member_register m ON l.member_id = m.id
JOIN loan_types lt ON l.loan_type_id = lt.loan_type_id
JOIN banks b ON l.bank_id = b.bank_id
WHERE l.status = 'pending'
ORDER BY l.date_apply ASC;

```

```

mysql> SELECT l.loan_id,
->      m.name AS member_name,
->      m.ic AS member_ic,
->      lt.loan_type,
->      l.loan_amount,
->      l.monthly_gross_salary,
->      l.monthly_net_salary,
->      l.monthly_repayment,
->      l.date_apply,
->      b.bank_name,
->      b.bank_account
-> FROM loans l
-> JOIN member_register m ON l.member_id = m.id
-> JOIN loan_types lt ON l.loan_type_id = lt.loan_type_id
-> JOIN banks b ON l.bank_id = b.bank_id
-> WHERE l.status = 'pending'
-> ORDER BY l.date_apply ASC;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| loan_id | member_name | member_ic | loan_type | loan_amount | monthly_gross_salary | monthly_net_salary | monthly_repayment | date_apply | bank_name |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| LOAN-1737940292 | TAN ZHI MING | 041118011233 | Skim Khas | 7000 | 8000 | 7600 | 716.14 | 2025-01-27 | AmBank (M) Berhad |
| 0987654321 | | | | | | | | | |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

2. UPDATE: Approve a loan application

```

UPDATE loans
SET status = 'approved',
    updated_at = CURRENT_TIMESTAMP
WHERE loan_id = 'LOAN-1737940292'
AND status = 'pending';

```

```
mysql> UPDATE loans
-> SET status = 'approved',
->    updated_at = CURRENT_TIMESTAMP
-> WHERE loan_id = 'LOAN-1737940292'
-> AND status = 'pending';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

	loan_id	member_id	loan_type_id	bank_id	date_apply	loan_amount	interest_rate	monthly_repayment	monthly_gross_salary	monthly_net_salary	loan_period	status	created_at	updated_at
Delete	LOAN-1737940292	1	3	1	2025-01-27	7000	5	716.14	8000	7600	10	approved	2025-01-27 09:11:32	2025-01-27 09:34:29

3. *INSERT: Record loan disbursement transaction*

```
INSERT INTO transactions (
    member_id,
    type,
    loan_id,
    amount,
    created_at,
    updated_at
)
SELECT
    l.member_id,
    'loan',
    l.loan_id,
    100.00,
    NOW(),
    NOW()
FROM loans l
WHERE l.loan_id = 'LOAN-1737940292'
AND l.status = 'approved'
AND l.member_id = 1;
```

```
mysql> INSERT INTO transactions (
->     member_id,
->     type,
->     loan_id,
->     amount,
->     created_at,
->     updated_at
-> )
-> SELECT
->     l.member_id,
->     'loan',
->     l.loan_id,
->     100.00,
->     NOW(),
->     NOW()
-> FROM loans l
-> WHERE l.loan_id = 'LOAN-1737940292'
-> AND l.status = 'approved'
-> AND l.member_id = 1;
Query OK, 1 row affected (0.00 sec)
Records: 1  Duplicates: 0  Warnings: 0
```

	id	member_id	type	savings_type	loan_id	amount	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	1	loan	NULL	LOAN-1737940292	100.00	2025-01-27 09:43:21	2025-01-27 09:43:21

4. UPDATE: Approve member registration

UPDATE member_register

SET status = 'approved',

updated_at = CURRENT_TIMESTAMP

WHERE id = 2

AND status = 'pending';

```
mysql> UPDATE member_register
-> SET status = 'approved',
->     updated_at = CURRENT_TIMESTAMP
-> WHERE id = 2
-> AND status = 'pending';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

SELECT * FROM 'member_register'

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	created_at	updated_at	no.	name	email	ic	phone	address	city	postc	state	gei	DC	agama	bangsa	no_pf	sa	ofi	ofi	ofi	ofi	gu	status
☐ Edit ☐ Copy ☐ Delete	1	2025-01-27 09:06:45	2025-01-27 09:07:48	A2:	TAN ZHI MING	bryanzhir	04	01116596391	UTM KL	JOHOR	8404i	Johor	Lel	20	Buddha	Cina	1928	80i	UT JB	Pe Jar	90i	WP: Kual Lum	2	approved
☐ Edit ☐ Copy ☐ Delete	2	2025-01-27 09:51:49	2025-01-27 09:52:32	A2:	LEE MING	brayantan	04	0195729461	UM KL	JOHOR	8404i	Johor	Lel	20	Kristan	Cina	2315	80i	Ue To	Ge To	86i	Pula Pina	3	approved

☐ Check all | With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

5. JOIN Query: Get all approved members with their active loans

```
SELECT m.id, m.name, m.ic, m.email,
       COUNT(l.loan_id) as active_loans,
       SUM(l.loan_amount) as total_loan_amount
FROM member_register m
LEFT JOIN loans l ON m.id = l.member_id
       AND l.status = 'approved'
WHERE m.status = 'approved'
GROUP BY m.id, m.name, m.ic, m.email
ORDER BY m.id;
```

```
mysql> SELECT m.id, m.name, m.ic, m.email,
-> COUNT(l.loan_id) as active_loans,
-> SUM(l.loan_amount) as total_loan_amount
-> FROM member_register m
-> LEFT JOIN loans l ON m.id = l.member_id
-> AND l.status = 'approved'
-> WHERE m.status = 'approved'
-> GROUP BY m.id, m.name, m.ic, m.email
-> ORDER BY m.id;
```

id	name	ic	email	active_loans	total_loan_amount
1	TAN ZHI MING	041118011233	bryanzhiming@gmail.com	1	7000
2	LEE MING	041118011234	brayantan123321@gmail.com	0	NULL

2 rows in set (0.00 sec)

6. JOIN Query: View transaction history at individual report

```
SELECT
  t.*,
  CASE
    WHEN t.type = 'savings' THEN 'Simpanan'
    WHEN t.type = 'loan' THEN 'Bayar Balik'
    ELSE t.type
  END as type_display,
  CASE
    WHEN t.type = 'loan' THEN CONCAT('LOAN-', l.loan_id)
    ELSE '-'
  END as reference
FROM transactions t
LEFT JOIN loans l ON t.loan_id = l.loan_id
WHERE t.member_id = 1
ORDER BY t.created_at DESC;
```

```
mysql> SELECT
-> t.*,
-> CASE
-> WHEN t.type = 'savings' THEN 'Simpanan'
-> WHEN t.type = 'loan' THEN 'Bayar Balik'
-> ELSE t.type
-> END as type_display,
-> CASE
-> WHEN t.type = 'loan' THEN CONCAT('LOAN-', l.loan_id)
-> ELSE '-'
-> END as reference
-> FROM transactions t
-> LEFT JOIN loans l ON t.loan_id = l.loan_id
-> WHERE t.member_id = 1
-> ORDER BY t.created_at DESC;
```

id	member_id	type	savings_type	loan_id	amount	created_at	updated_at	type_display	reference
1	1	loan	NULL	LOAN-1737940292	100.00	2025-01-27 09:43:21	2025-01-27 09:43:21	Bayar Balik	LOAN-LOAN-1737940292

7. JOIN Query: Get loan approval statistics by loan status

```

SELECT lt.loan_type,
       COUNT(l.loan_id) as total_applications,
       SUM(CASE WHEN l.status = 'approved' THEN 1 ELSE 0 END) as approved_count,
       SUM(CASE WHEN l.status = 'rejected' THEN 1 ELSE 0 END) as rejected_count,
       AVG(CASE WHEN l.status = 'approved' THEN l.loan_amount ELSE 0 END) as
       avg_approved_amount
FROM loan_types lt
LEFT JOIN loans l ON lt.loan_type_id = l.loan_type_id
GROUP BY lt.loan_type_id, lt.loan_type
ORDER BY total_applications DESC;

```

```

mysql> SELECT lt.loan_type,
-> COUNT(l.loan_id) as total_applications,
-> SUM(CASE WHEN l.status = 'approved' THEN 1 ELSE 0 END) as approved_count,
-> SUM(CASE WHEN l.status = 'rejected' THEN 1 ELSE 0 END) as rejected_count,
-> AVG(CASE WHEN l.status = 'approved' THEN l.loan_amount ELSE 0 END) as avg_approved_amount
-> FROM loan_types lt
-> LEFT JOIN loans l ON lt.loan_type_id = l.loan_type_id
-> GROUP BY lt.loan_type_id, lt.loan_type
-> ORDER BY total_applications DESC;

```

loan_type	total_applications	approved_count	rejected_count	avg_approved_amount
Skim Khas	1	1	0	7000
Al-Bai	0	0	0	0
Al-Inah	0	0	0	0
Karnival Muslim Istimewa	0	0	0	0
Baik Pulih Kenderaan	0	0	0	0
Cukai Jalan	0	0	0	0

6 rows in set (0.00 sec)

8. SELECT: Get pending loan applications with member details for approval

```

SELECT l.loan_id,
       m.name AS member_name,
       m.ic AS member_ic,
       lt.loan_type,
       l.loan_amount,
       l.monthly_gross_salary,
       l.monthly_net_salary,
       l.monthly_repayment,
       l.date_apply,
       b.bank_name,
       b.bank_account
FROM loans l
JOIN member_register m ON l.member_id = m.id
JOIN loan_types lt ON l.loan_type_id = lt.loan_type_id
JOIN banks b ON l.bank_id = b.bank_id
WHERE l.status = 'pending'
ORDER BY l.date_apply ASC;

```

```

mysql> SELECT l.loan_id,
->      m.name AS member_name,
->      m.ic AS member_ic,
->      lt.loan_type,
->      l.loan_amount,
->      l.monthly_gross_salary,
->      l.monthly_net_salary,
->      l.monthly_repayment,
->      l.date_apply,
->      b.bank_name,
->      b.bank_account
-> FROM loans l
-> JOIN member_register m ON l.member_id = m.id
-> JOIN loan_types lt ON l.loan_type_id = lt.loan_type_id
-> JOIN banks b ON l.bank_id = b.bank_id
-> WHERE l.status = 'pending'
-> ORDER BY l.date_apply ASC;

```

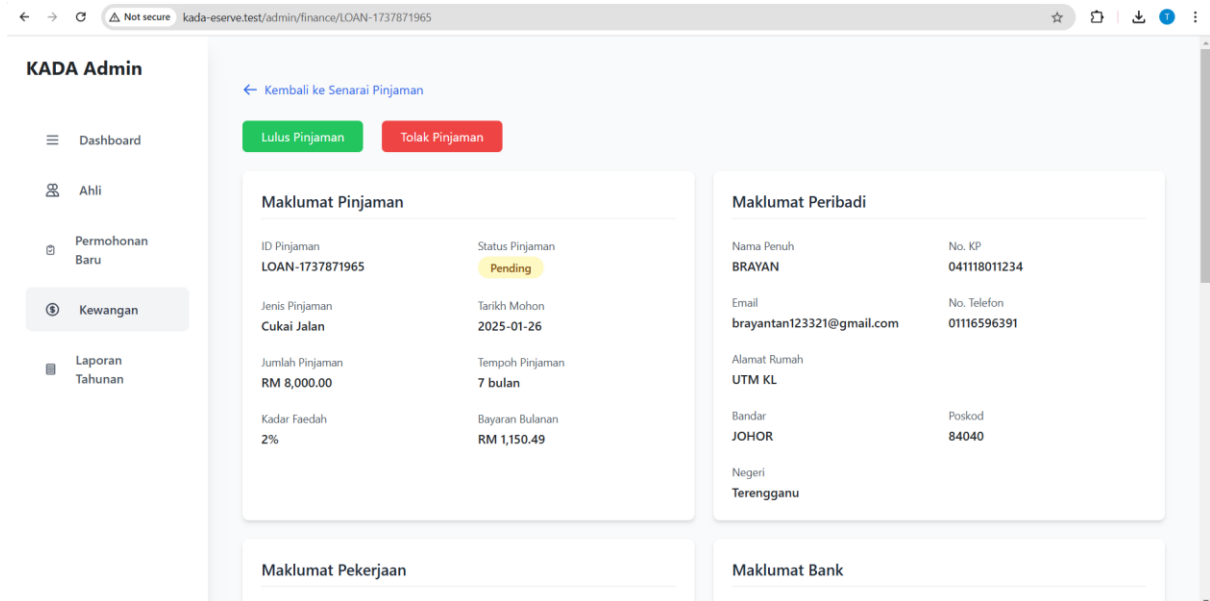
loan_id	member_name	member_ic	loan_type	loan_amount	monthly_gross_salary	monthly_net_salary	monthly_repayment	date_apply	bank_name
LOAN-1737943229	TAN ZHI MING	041118011233	Al-Bai	3000	80000	7500	1008.34	2025-01-27	Affin Islamic Bank Berhad

```

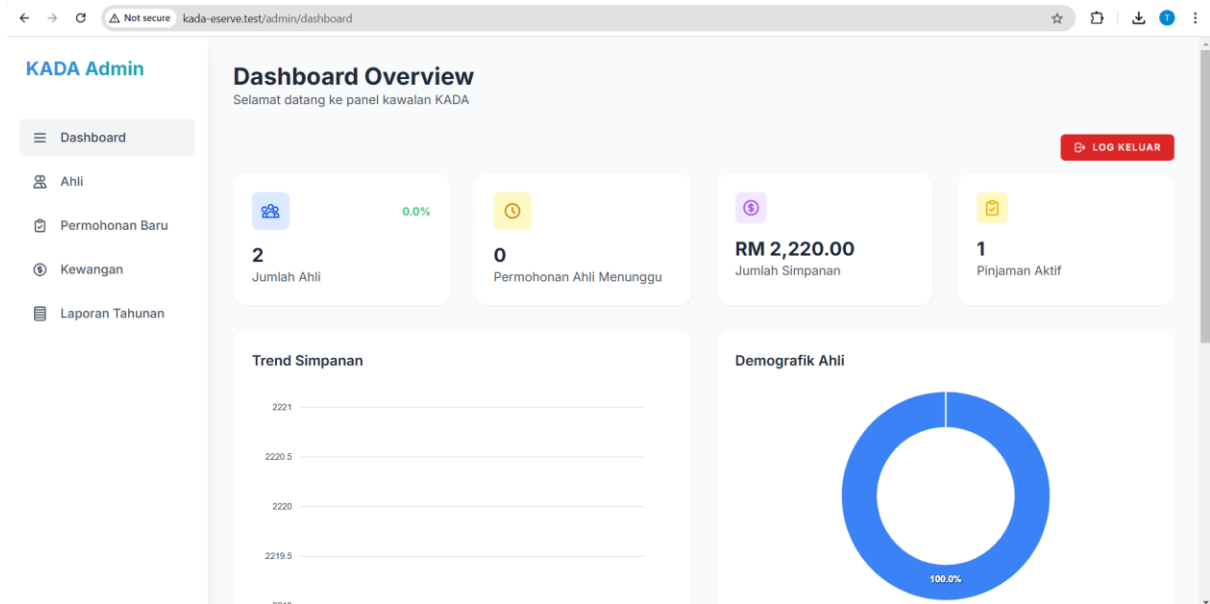
1 row in set (0.00 sec)

```

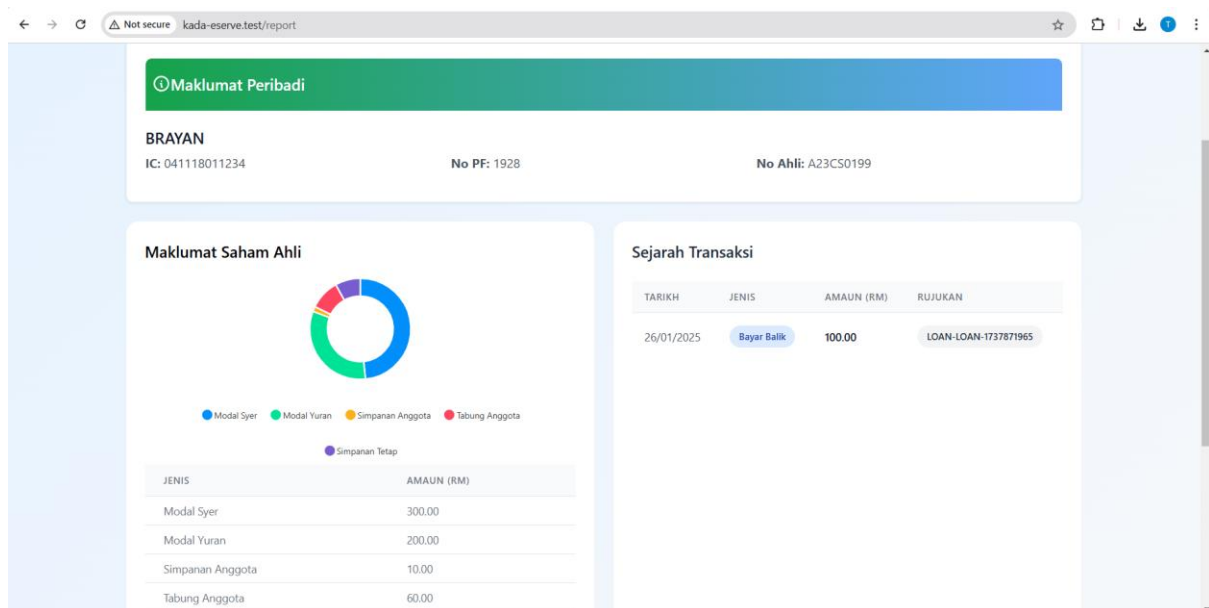

3.2 Interface Implementation



This interface is related with the 8th DML SQL queries. This page fetch the data of the selected member from 4 table which is member_register , loans , banks and laon_types. This interface is to get pending loan applications with member details for approval.



This interface related to 7th in DML SQL queries. This SQL queries is to get loan approval statistic by loan status. The pinjaman aktif reflect the total number of loan with status approved.



This interface is page that for member to view the transaction record. This feature used the 6rd SQL queries from DML. It show the transaction activities of the logged in account member.

SECTION D (CONCLUSION AND REFLECTION)

4.1 Conclusion

The proposed administration module is designed to provide administrators with enhanced capabilities, including loan application and member registration approval, seamless transaction management, and robust statistical dashboards for analytical purposes. These functions are supported by efficient implementation of SQL statements, thus making data processing both accurate and efficient. A key factor contributing to the effectiveness of the module is the design of a clear and well-structured database. By prioritizing database optimization and organization, the system minimizes redundancy, reduces processing time, and ensures the reliability of its operation. As such, the module is positioned to streamline administrative workflows, improve decision making through data visualization, and enhance the overall user experience. This integration of automated tools and structured database systems shows the potential of technology to transform traditional management processes, paving the way for more efficient and error-free operating frameworks.

4.2 Reflection

In the group, I took the lead role in system development, while my teammate Sabrina took the project lead for the database initiative. Through this project, I realized that creating an optimized system relies heavily on a well-structured database. Since this was our first project, we encountered challenges due to our limited experience with database concepts. This often results in the need to modify the database repeatedly, either to add missing tables, or to correct

incorrect join relationships. However, when we take an agile approach, these changes are manageable and less costly.

My main contribution to the database consisted of coming up with the necessary attributes and tables, which the team then discussed and normalized together. In addition, I also prepared a data dictionary. At times, I was frustrated because our initial database design was immature, leading to inconsistencies and resulting in contradictions in the project. Sometimes, proposed database ideas lack thorough analysis, leading to problems that affect the entire team due to inconsistent data structures. Despite these difficulties, I am grateful to have a supportive team. Working together to solve these problems reinforces the importance of teamwork and communication.

When modifying the database, we sometimes have to pause the current task to resolve these issues before continuing, to ensure that the error is not repeated. This experience highlights the importance of thorough pre-documentation and requirements analysis before beginning implementation. However, I've also learned that adaptability is key to team projects. A good project team should be open to changes and deal with them calmly and constructively.

From this experience, I learned that future projects must prioritize solid requirements analysis and foster an adaptable and collaborative team culture. To improve my skills, I plan to study more project source code to better understand the typical data structures required by various systems. Familiarizing myself with these examples will give me a clearer idea of what data is typically required for a complete system. This preparation will help me build a stronger database and contribute more effectively to future projects.